MACHINE LEARNING ENGINEER PORTFOLIO · NOV 2026

Sungwon (Alex) Lee

이성원

Machine learning engineer who builds forecasting models and carries them into production

I studied statistics and machine learning at UCLA and Carnegie Mellon. At Herit, an energy company in Korea, I build electricity demand models and deploy them to the service servers. I spend as much time checking that offline numbers hold up in the production path as I do choosing models.

Email sungwonlee.alex@gmail.com

LinkedIn linkedin.com/in/sungwonalexlee

CONTENTS

01 About 02

02 Experience & Education 03

03 Skills 04

04 LH Homeniq Load Forecasting 05
Herit · main project

05 Lost in Quantization 10
AAAI-27 workshop paper

06 Korean Equity Return Prediction 12
Personal project

07 Other Projects 13

01 About

I hold a BS in Statistics and Data Science from UCLA and an MS in Data Analytics for Science from Carnegie Mellon. Since July 2025 I have worked at Herit as a data scientist under Korea's alternative military service program, on apartment-complex load forecasting, an AI agent for demand-response customer support, and automated power-market bidding.

My longest project forecasts next-day electricity use for 77 LH Homeniq apartment complexes. Training and inference had to run on the service server itself (Python 3.7, 8 CPU cores, no GPU). The final model matches Chronos-2, a time-series foundation model that cannot be installed on that server (R^2 0.879 vs 0.881).

Education	MS Data Analytics for Science, Carnegie Mellon University (May 2024, GPA 4.06/4.33) BS Statistics and Data Science, UCLA (June 2023)
Service	Technical Research Personnel (alternative military service), since July 2025
Languages	Korean (native) · English (fluent; BS and MS in the US, two US internships)

I compare like with like

Every model comparison pairs the same complex and the same hour, and reports MAE and R^2 together. On the LH project, one week of results made the network look tied with the baseline (MAE −1.0%). The cause was two code paths computing the baseline differently. Measured through the same path over 30 days, the network's MAE was 22% lower.

I distrust good numbers first

I traced a 17% gap between offline and production-replay accuracy to future information leaking through preprocessing, and fixed it. In a personal equity project, I re-tested my own results and removed t-statistics inflated 3–4× by overlapping samples and a survivorship-biased universe.

I look at the data before the architecture

Changing the LH network's width, depth and input length mostly did nothing or hurt. What moved accuracy was rebuilding the training set point-in-time and backfilling two years of weather observations and forecasts from an internal API.

02 Experience & Education

Jul 2025 - now

Herit Corp. Data Scientist (Manager) · Technical Research Personnel

- Built next-day load forecasting for 77 LH Homeniq apartment complexes and handed it off to the service team → 04
- Deployed energy-use forecasting for 1,000+ households at Raemian One Perla, with alerts before billing-tier thresholds
- Built HERIA, an AI agent that automates customer (VoC) responses for the demand-response service; response time improved 60%
- Built an automated power-trading platform that uses forecasting and reinforcement learning to optimize KPX bidding

Sep - Dec 2024

Ferveret Inc. Data Science Intern · Remote (San Jose, CA)

- Built a model predicting data-center cooling power use; energy efficiency improved 15%
- Found operating patterns behind a 10% cost reduction in data-center data; presented to senior engineers

Jun - Aug 2024

PricewaterhouseCoopers Deals Consulting Intern · San Francisco, CA

- Analyzed M&A contract separation; helped design a workforce scenario-planning tool (12% lower overhead)

Jan - May 2024

Ansys × CMU Capstone Data Scientist · Pittsburgh, PA

- Built an RNN forecasting cloud-simulation memory use; simulation accuracy improved 20%
- Optimized the cloud-simulation data pipeline; compute time down 30%

Education

Carnegie Mellon University
MS Data Analytics for Science · GPA 4.06/4.33
May 2024

University of California, Los Angeles
BS Statistics and Data Science · major GPA 3.60/4.00
June 2023

Research

Lost in Quantization
Submitted to AAAI-27 Workshop on Deployable AI → 05
Nov 2026

Activities

Graduate TA, CMU School of Computer Science (Communications for Software Leaders II) 2024

CFO, Google Developer Student Club at CMU 2023-2024

Alternate captain, Korea U-18 national ice hockey team (2018 IIHF U18 World Championship, Div II A) · UCLA ice hockey (ACHA D2)

03 Skills

Area	Tools	Where I used them
Modeling	PyTorch · TensorFlow 2 / Keras · scikit-learn · LightGBM · Hugging Face Transformers	LH forecasting network (TF 2.5, five custom layers, shipped as .h5) and HGB · partial fine-tuning of CoGenCast
Time series	Quantile regression · Chronos-2 · TimesFM 2.5 · interval calibration · CRPS	Foundation models as the accuracy reference · 80% interval coverage calibrated from 72.9% to 78.8%
LLMs	vLLM · NVFP4 / ModelOpt quantization · Ollama · JSON-schema-constrained output	Quantization safety paper (2,014 request pairs × 3 models) · local voice assistant
Data	pandas · NumPy · SQL · Spark · Parquet	15-minute meter data into point-in-time panels · two-year backfill from an internal weather API
Production	Linux · cron batch jobs · conda / uv · fallbacks · run manifests · retraining gate	LH serving: 16 s per daily run on 8 CPU cores, tiered fallback on failure
Evaluation	Paired comparison (MAE · R^2 · win/loss) · bootstrap CIs · Newey–West t · human-label validation	Model selection in every project · validating LLM judges against blind human labels
GPU (learning)	CUDA · Triton · PyTorch C++ extensions (TORCH_LIBRARY)	Kernel exercises · an internal extension template and its benchmark
Other	R · MPI · MLflow · Azure · Tableau	Graduate coursework and internships

04 LH Homeniq Apartment Load Forecasting

Period Aug – Nov 2026 **Company** Herit **Role** Sole engineer: data cleaning, modeling, evaluation, serving code, handoff

Stack TensorFlow 2.5 · scikit-learn HGB · pandas · Chronos-2 (reference)

Problem

Forecast next-day hourly electricity use for the 77 apartment complexes in LH's Homeniq service. The first forecast is published at 06:00 the day before; a second at 02:00 on the day, using data through the previous night.

Constraints

1. Training, inference and maintenance all on **the service server itself** (Python 3.7 · TensorFlow 2.5 · 8 CPU cores, no GPU)
2. Forecast **every complex, including 47 with intermittent metering** (30 complete + 47 partial)
3. Deployment bar: **mean $R^2 \geq 0.82$** across complexes, and better than the incumbent model

Results

Day-ahead forecast, 06:00

0.879 R^2

MAE 2.61 kW

Same-day forecast, 02:00

0.898 R^2

MAE 2.37 kW (−9% vs day-ahead)

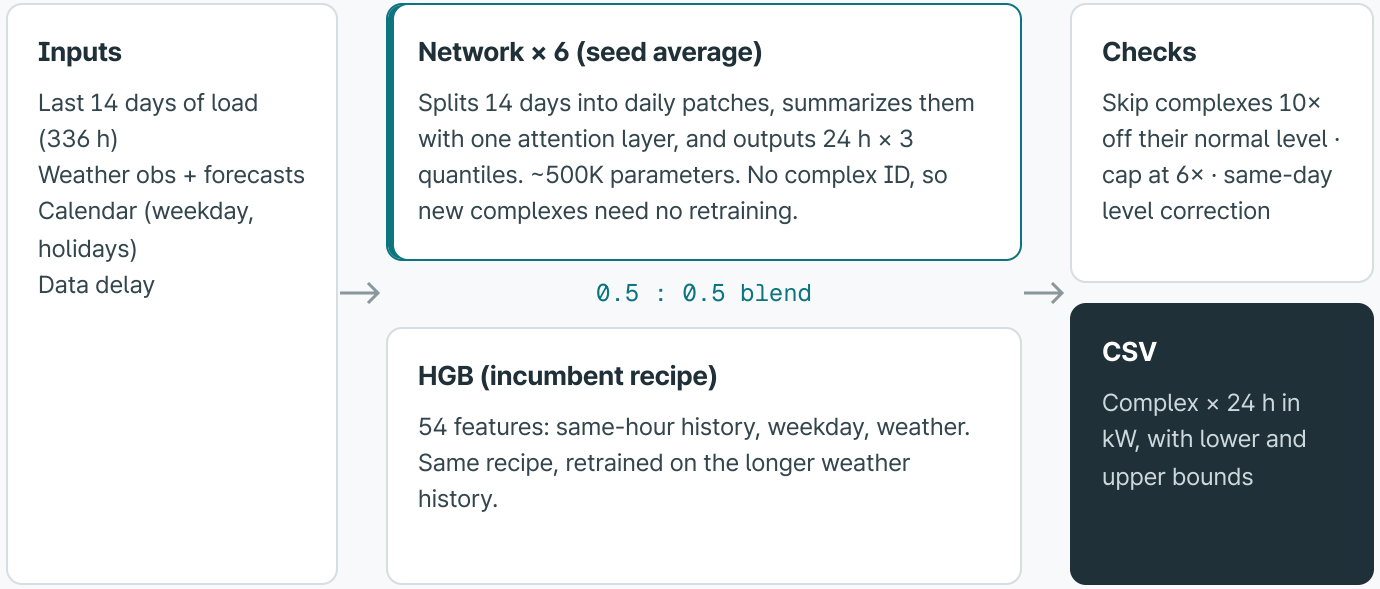
74 complexes · matched hours	Mean R^2	$R^2 \geq 0.82$	MAE
Same-day, 02:00	0.898	64	2.37 kW
Day-ahead, 06:00	0.879	63	2.61 kW
Chronos-2 day-ahead (can't run on server)	0.881	63	2.67 kW

Against the incumbent model (HGB): R^2 **0.842 → 0.884**, MAE **3.03 → 2.56 kW** (68 complexes). A daily run takes **16 s** and 1.4 GB on 8 cores.

Scored by replaying the production scripts over 210 days: January, February, spring (March–June) and August.

The best model couldn't be deployed, so it became the yardstick

1. Zero-shot Chronos-2 was the most accurate, with about 30% lower MAE than the incumbent in August. It needs Python 3.9 or later; the server runs 3.7.
2. An ONNX export loaded under Python 3.7 onnxruntime, but adding a new runtime to the server was rejected as an operational burden. Distillation was ruled out because retraining had to happen on the server, where the teacher cannot run.
3. Chronos-2 stayed on as **the yardstick for the remaining gap**. I designed a small network that trains, saves and loads in the server's TF 2.5, and publish a 50:50 blend with the incumbent-style HGB.



Why blend

The network reads raw series and HGB reads engineered features, so they fail differently. I tried weights from 0.3 to 0.8; 0.5 was best.

Spike-proof scaling

Per-sample scaling uses median and IQR instead of mean and SD, clipped at ±8. Added after one meter spike blew up a forecast.

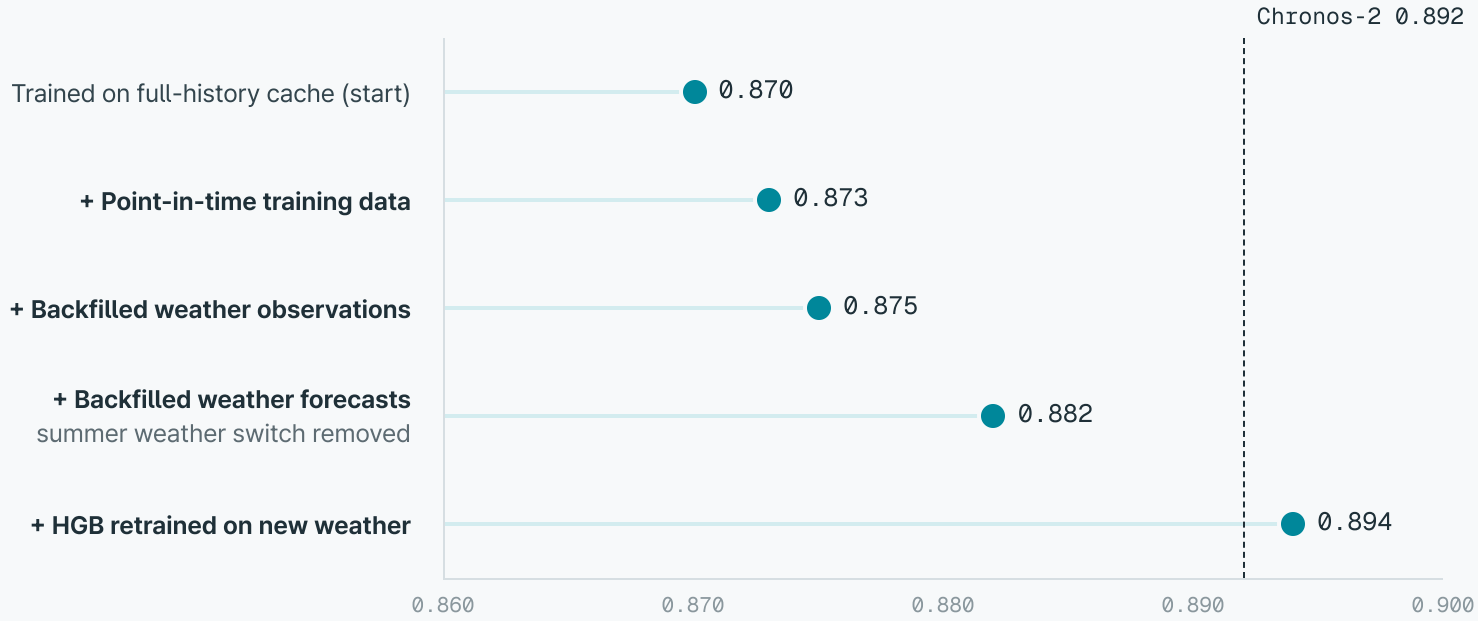
Old-stack workarounds

Keras LayerNormalization breaks under TF 2.5 with NumPy 1.21, so I wrote my own. Layers holding constants became custom layers so the model saves as .h5.

Accuracy came from data work, not model settings

R^2 as each data fix was added (day-ahead, blended)

Chronos-2 comparison set of 42 complexes · matched hours · production replay over four periods



The axis starts at 0.860. Because design and scoring used the same four periods, the final report compared all complexes, partial ones included (0.879 vs 0.881), and called the result on par.

Model changes that didn't help

- Network width 32 · 48 · 96 · 128; number of blocks
- Input length of 28 or 56 days (14 was best)
- Dropout, batch size, early-stopping criteria
- Only going from 3 to 6 seeds helped, slightly

Settings with a better early-stopping loss scored worse on the test periods, and changing the validation scheme didn't change that. I concluded the model-side knobs were used up for this data.

Why weather hurt in summer

The weather history covered only a few winter months, so the model learned the heating relationship (load −0.4% per °C) and applied it backwards in the cooling season (+2.8% per °C). I first switched weather off by season, using a rule fixed before looking at results and validated on unseen months. After backfilling forecasts from September 2024, the rule was no longer needed.

Problems caught before deployment

LEAKAGE

Offline accuracy was 17% better than production replay

Cause Training windows were cut from a full-history cache whose gap-filling looks both ways, so values from after the information cutoff leaked into inputs. On average 6.4% of the last 24 input hours differed from production.

Fix Rebuilt the panel for every date from only the raw data available at that time: 31,377 samples, 40 minutes on 4 processes.

Result In spring, the blend went from trailing HGB to beating it. The same check on HGB showed no effect (R^2 difference 0.000), so HGB stayed as is.

ROBUSTNESS

One meter spike drove a complex's R^2 to −881

Cause A 15-minute reading about 1,290× normal passed outlier filtering and stayed in the inputs for 14 days, inflating the sample SD from 9 to 809.

Fix Median/IQR scaling, winsorizing at ±8, and a cap at 6× the input median. A later case of fully corrupted inputs (a 10.8 kW complex forecast at 13,454 kW) added a rule: skip any complex 10× off its normal level.

Result January mean R^2 for the network went from −19.26 to 0.781. Setting the bar on the mean, not the median, exposed it before launch.

COLD START

Late meter data and short histories

Cause Meter data arrives at different times per complex. With one shared cutoff, a replay day forecast only 29 of 77 complexes.

Fix Fed the delay to the model as an input and randomized it in training. Complexes with under 14 days of history are padded by repeating the days they have; only those cut off for over a month are skipped.

Result Coverage went from 74.1% to 77.4%. Most remaining exclusions have no ground truth either.

TRAINING

Three of six production models were saved mid-divergence

Cause Corrupted samples (536–2,882× normal) entered training and exploded the gradients. The same seed diverged in 3 of 5 runs, depending on thread order.

Fix Filter training samples with the same input checks used in serving (13 removed), warn on stops before epoch 30, and check each model's epochs and validation loss after retraining.

Result All six retrained models converged (42–67 epochs), and checking training logs became part of every model swap.

Operations and lessons

Daily run

1. **Collect weather**
   Observations and forecasts for 58 grid cells from the internal weather API (verified on Python 3.7)
2. **Build cutoff panels**
   Inputs per complex from raw data received up to 2 h before publication, with delay and short-history handling
3. **Infer and blend**
   Six networks + HGB, 16 s and 1.4 GB on 8 cores
4. **Publish**
   CSV plus a manifest (exclusion reasons, safeguards triggered); automatic scoring and alerts the next day

When something fails

HGB fails	Both fail	Same-day run fails
Publish the network alone	Same-hour median of the last 7 days (pandas only)	Keep the day-ahead forecast

Monthly retraining keeps the current model if the candidate is worse on the last two weeks by 2% MAE or 0.005 R^2, and any version can be rolled back.

Lessons

Agree on the evaluation set first

I first scored on the 42 complexes that had all five models and all four periods. The requirement was every complex, partial ones included, and switching to it exposed the coverage gap.

One season is not enough

Had I shipped on August results alone, winter MAE would have been 11% worse than the incumbent. Every decision is checked on January, February, spring and August.

Hand off for the receiver

At the service owner's request I cut the handoff package three times, down to models, weights and the I/O spec, and confirmed zero prediction difference in a clean Python 3.7 environment before handing it over.

What's left

Re-checking accuracy on September onward, which played no part in design, and the rollout to the production server.

05 Lost in Quantization

The Refusals a 4-bit Checkpoint Flips and Its Chat Template Hides

Period Oct – Nov 2026 **Type** Independent research · submitted to AAAI-27 Workshop on Deployable AI · first author **Stack** vLLM · NVFP4 · ModelOpt · 2× RTX 5060 Ti

Running an 8B model on one consumer GPU usually means downloading a 4-bit checkpoint that someone else quantized. I asked whether a downloaded NVFP4 checkpoint refuses the same harmful requests and answers the same benign ones as the original, whether that differs between English and Korean, and how reliably the difference can be measured.

2,014 paired requests (1,069 harmful · 945 benign, English and Korean) · three 8B models (Qwen3, Llama-3.1, EXAONE-3.5) · vendor and self-quantized checkpoints · two refusal judges validated against blind human labels

01 **The chat template moves refusals more than quantization.** Using the template shipped with NVIDIA's NVFP4 Llama-3.1-8B on the original BF16 weights raises the Korean refusal rate on harmful requests from 34.6% to 62.7%. Comparing the checkpoints as shipped hides a Korean regression.

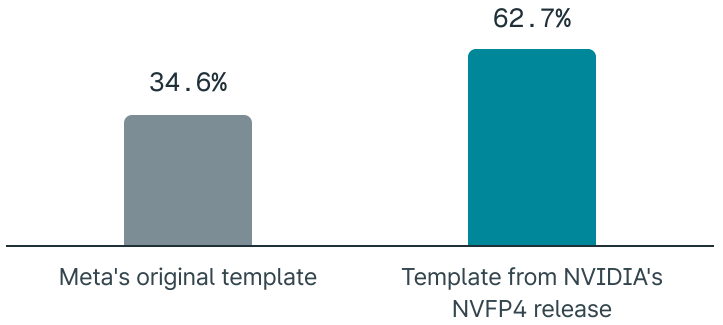
02 **Small net change, many flips.** Net refusal rates move by under 5 pp, but 5–10% of harmful requests flip from refusal to compliance, against 0.5–2.2% when BF16 is simply rerun.

03 **Korean flips more.** For the model with the largest English–Korean gap, Korean refusals flip 3–9 pp more often than English ones, and bilingual calibration does not reduce this.

04 **The evaluation setup is part of the result.** Splitting the same checkpoint across two GPUs flips up to 9% of Korean harmful requests, and the judges are least accurate on exactly the responses that flip.

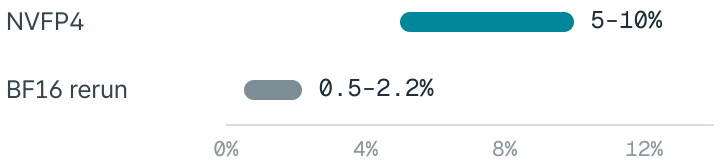
Korean refusal rate on harmful requests · Llama-3.1-8B BF16

Same weights, different chat template



Harmful requests flipped from refusal to compliance

Gemma-4-31B judge



Making the measurement trustworthy

BASELINE

Measure the noise floor first

- Rerunning BF16 with only the batch order shuffled gave a noise baseline. Every quantization effect is read against it.
- Even the noise flipped Korean refusals more often than English ones, so without the baseline a language-specific quantization harm could be misread.
- A 2×2 design (template × precision) separated the template confound from quantization.

JUDGES

Validate the LLM judges against people

- I built a blind labeling interface, labeled 330 responses myself, and estimated judge accuracy with inclusion-probability weights from a stratified sample.
- PolyGuard often read Korean compliance as refusal (32 of 142), so it was dropped as the primary judge.
- The rubric was frozen before seeing results, and low judge accuracy on flipped responses is reported as a limitation.

SYSTEMS

Make it run on consumer GPUs

- Got vLLM NVFP4 running on RTX 5060 Ti (sm_120) with a CUDA 12.8 driver by resolving wheel versions, the Triton ptxas path, the FlashInfer JIT cache and FP4 autotuner memory.
- With no P2P between the two GPUs, ran the large judge model with pipeline rather than tensor parallelism.
- 3,596 tok/s batch-64 decode for Qwen3-8B NVFP4 on a single GPU.

TAKEAWAY

Small differences in the deployment path, such as the chat template or how a model is split across GPUs, change outcomes as much as quantization itself. The paper shows in numbers why **a model should be evaluated as shipped and as served**. Every number in it is cross-checked against tables and experiment output by script.

06 Korean Equity Return Prediction: Stripping Out Inflated Results

Period Jun – Oct 2026 **Type** Personal project · Python package spred **Stack** pandas · LightGBM · PyTorch (LSTM, TFT) · Streamlit · FastAPI · cron

A package that ranks KOSPI stocks by 21-day forward return, with evaluation tooling for walk-forward splits, cross-sectional IC, long-short and long-only returns, transaction costs and market regimes.

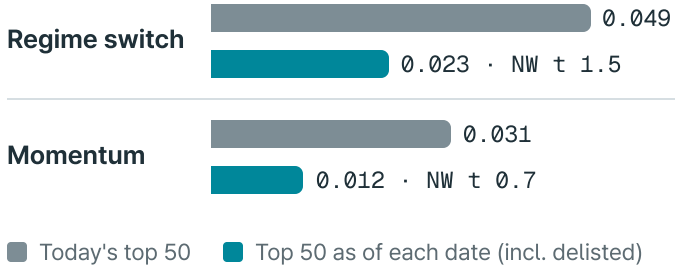
- **Six attempts that changed the model or signal mix** (Ridge and LightGBM cross-sectional models, a multi-factor composite, LSTM, flow-of-funds signals, TFT twice) all failed to beat plain 12-1 momentum. I concluded the bottleneck was the signal, not the model.
- A **regime-switching signal** (momentum when the index is above its 200-day average, a defensive composite below) was the first to beat momentum.
- Found and fixed a **leak** where future returns reached the sequence encoder, which had produced an IC of 0.935.

What the robustness check removed

- **Overlapping samples:** daily-sampled 21-day targets inflated t-statistics 3–4× (naive t 5–10 → Newey–West t 1.5–2.8).
- **Survivorship bias:** re-ran on the top 50 stocks by market cap as of each date, including 112 later-delisted names, instead of today's top 50.
- **What survived:** regime switching beats momentum in all three universes (IC difference NW t 3.3 / 1.7 / 2.7). Evidence of alpha did not.

Cross-sectional IC by universe construction

Jul 2015 – Sep 2026 · 21-day returns



More than half of the attractive number came from how the sample was built. Since then, the better a result looks, the sooner I check the sample and look for leaks.

07 Other Projects

TIME SERIES

Reproducing and debugging CoGenCast

Jun 2026 · LLM + flow-matching time-series model

- Full fine-tuning on a 16 GB GPU ran out of memory (optimizer state alone was ~14 GB). Freezing both Qwen3 copies except their last 4 layers brought it to ~135M trainable parameters and ~7 GB.
- Validation loss was stuck at 318: on constant 96-hour windows, normalization divided by √1e-5 and scaled inputs 316×. A floor on the SD fixed it; ETTh1 MSE went from 1.17 to 0.342.
- On Samsung Electronics daily returns, MSE was worse than predicting zero, but CRPS beat a rolling-volatility Gaussian (0.936 vs 0.955).

PyTorch · Transformers · CRPS

LLM AGENT

A free, local voice assistant

Oct 2026 · Ollama qwen2.5:14b + browser speech APIs

- When it called tools directly, the model filled nested schemas wrongly and returned empty answers to place-name questions.
- The model now fills only a JSON-schema-constrained plan form (place, topic), and code makes the tool calls.
- Model-written answers dropped numbers or slipped Chinese into Korean, so default answers come from code templates. Model polishing is optional and reverts when Chinese characters appear.

Ollama · JSON Schema · MCP · Web Speech

GPU

GPU kernel study and an extension template

Jun 2026 – · RTX 5060 Ti

- Writing matmul, softmax and FlashAttention kernels in CUDA and Triton to learn the GPU memory hierarchy and performance measurement.
- Built a PyTorch extension template for colleagues with TORCH_LIBRARY and register_autograd, so kernels work with backward and torch.compile (the pybind approach breaks both).
- In benchmarks a fused bias + GELU kernel tied torch.compile in fp32 and ran at 0.75–0.92× in bf16, so I decided not to ship it.

CUDA · Triton · torch.compile

Thank you for reading.

I'm happy to walk through the design decisions and experiment logs behind any of these projects in detail.

Name	Sungwon (Alex) Lee · 이성원
Email	sungwonlee.alex@gmail.com
LinkedIn	linkedin.com/in/sungwonalexlee